Activity 3 Report: Cross-Group Deployment and Verification

COMP 018 Database Administration | Project tested: PUP-SPOT Campus Lost and Found (PHP + PostgreSQL) | Date of testing: 6 November 2026

1. Cover and links

Item	Value
Testing group	G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph
Project tested	PUP-SPOT Campus Lost and Found
Owning group	Group 05 (grp05-dba)
Repository	https://github.com/grp05-dba/grp05_lostandfound
Tracker	https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests
Evidence files	a3_deploy.txt, a3_app.txt, screenshots of the public and admin pages

Only the repository and its documentation were used. Setup was done on the tester's own PostgreSQL server.

2. Test environment

Item	Value
Operating system	Windows 11
PostgreSQL	PostgreSQL 18.6 (x86_64 Windows), psql on PATH
PHP	PHP 8.2.12 (CLI, ZTS), extensions pdo_pgsql and pgsql loaded. The README asks for PHP 8.0 or later
Other tools	git 2.56.0, curl for the API checks, a web browser for the two pages
Database used	grp05_lostandfounddb (the name used in the README)

3. Installation procedure and result

Step	Action	Result
1	Extracted the zip and read README.md and docs/	Clear step-by-step guide. The README clones from github.com/grp05-dba/grp05_lostandfound
2	CREATE DATABASE grp05_lostandfounddb	OK
3	Ran sql/grp05_lostandfounddb_script.sql	OK, 8 tables created, no errors
4	Ran the necessary-data script	OK, 11 categories inserted
5	Ran the role script after replacing both enter_your_password_here values with test passwords	OK, roles lostandfound_app and lostandfound_public created with their grants

6	Ran the demo data script	OK, 6 items, 7 lost reports, 3 claims and 6 potential matches. The script is rerunnable
7	Copied .env.example to .env and filled the database settings	OK. The EmailJS and TextBee keys were left empty, as the README allows
8	Started php -S on ports 8000 (public) and 8001 (admin) as described	Both servers started. Both app.html pages returned HTTP 200

4. Database verification

After the scripts: 8 tables (categories, items, lost_reports, claims, potential_matches, item_images, lost_report_images, claim_images) and 53 constraints (8 primary keys, 8 foreign keys, 2 unique, 8 check, 27 not null). The database has no views and no triggers. Two login roles exist, neither is a superuser, both have a password. Counts come from a3_deploy.txt.

5. Application checks

Public interface (port 8000) and admin interface (port 8001) were used in a browser. The API was also called with curl.

Area	Action tested	Result
Public	Browse the found items list (Figures 1 and 2)	Pass. Shows the 5 unclaimed items
Public	Submit a lost report with a photo (Figures 3 and 4)	Partial. The report is saved (LR-2026-516703) but the page shows no reference code (issue 1)
Public	Track a report and a claim by code (Figures 5 and 6)	Pass. The report page also lists a potential match
Public API	Categories, items, search, add report, view report, add claim, view claim	Pass. All records were created and read back
Public API	Missing fields, unknown code, empty body	400 with a clear message, 404, and a PHP fatal error (issue 2)
Admin	Overview, found inventory, lost reports, claim verification (Figures 7 to 10)	Pass. Opens with no login
Admin API	List items, reports and claims, approve a claim, add an item	Pass. Claim became approved, new item created (HTTP 201)
Admin API	Gateway statistics	Responds with "Unable to connect to TextBee" because no key is set
Both	Which database role each server uses	Public server uses lostandfound_public, admin server uses lostandfound_app (proved by disabling each role in turn)

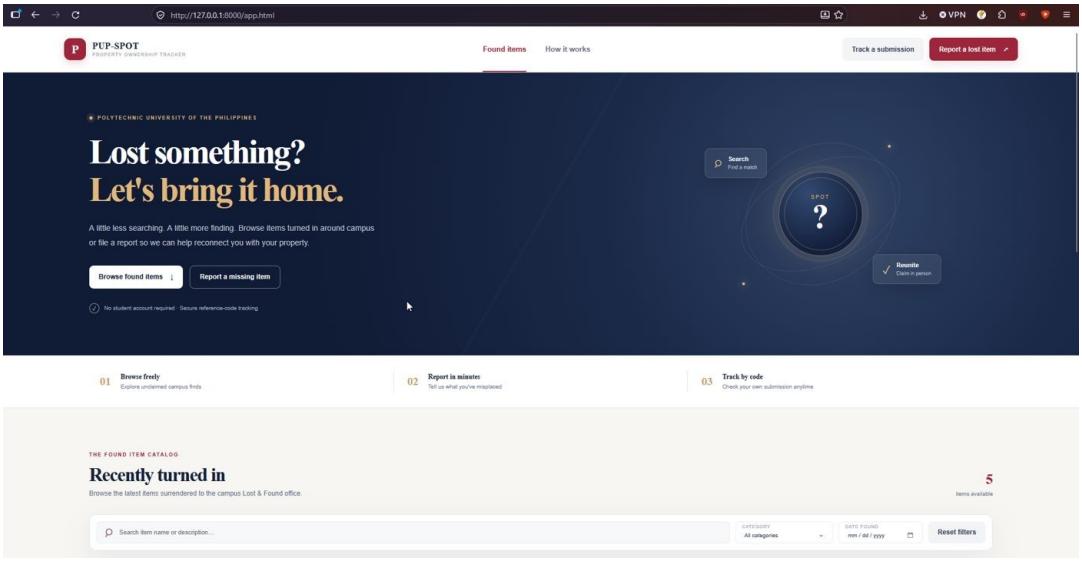


Figure 1. Public home page.

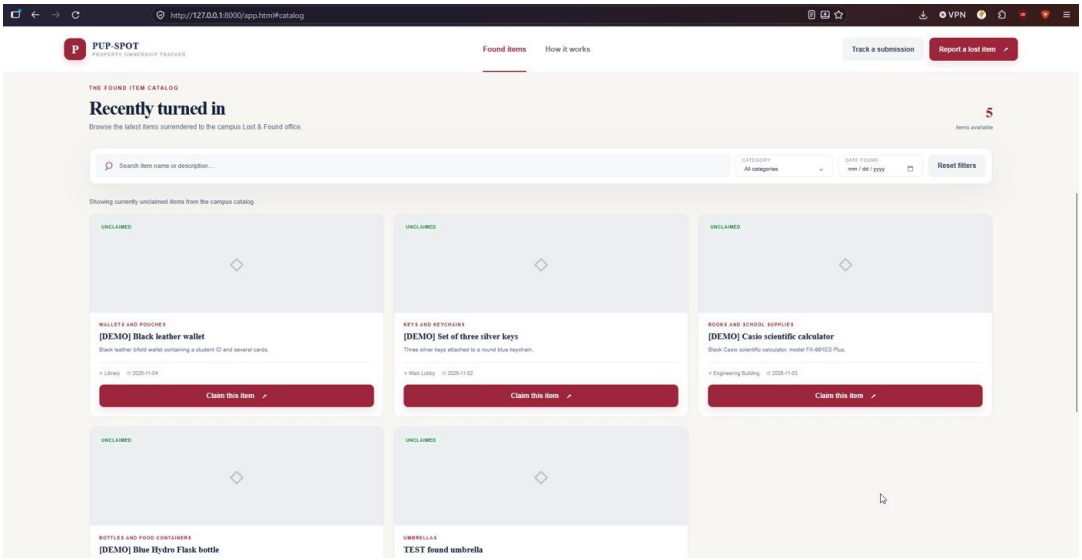


Figure 2. Public found items catalog (demo data plus one test item).

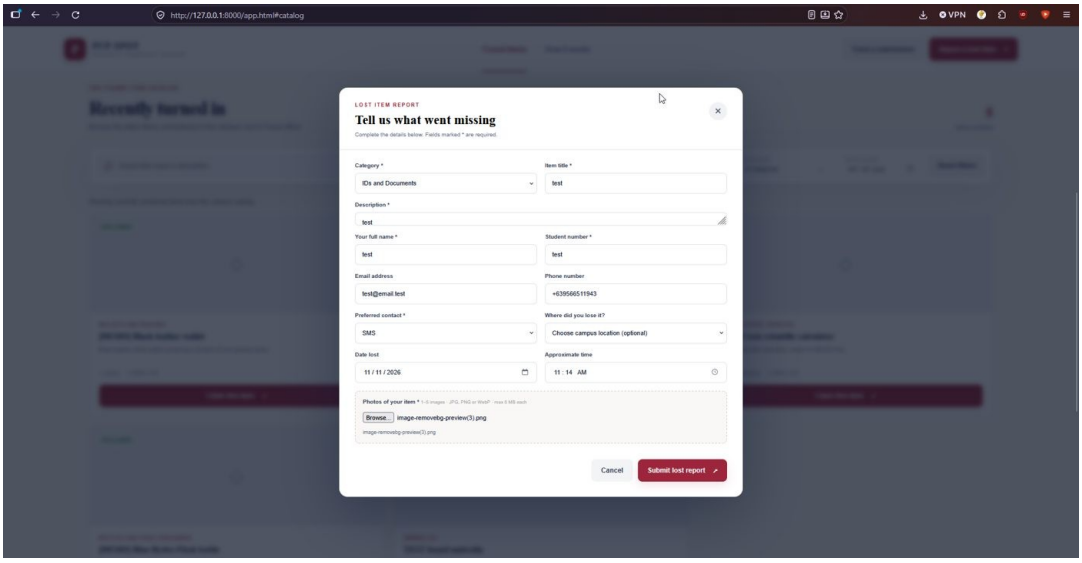


Figure 3. Lost report form filled with test data and a photo.

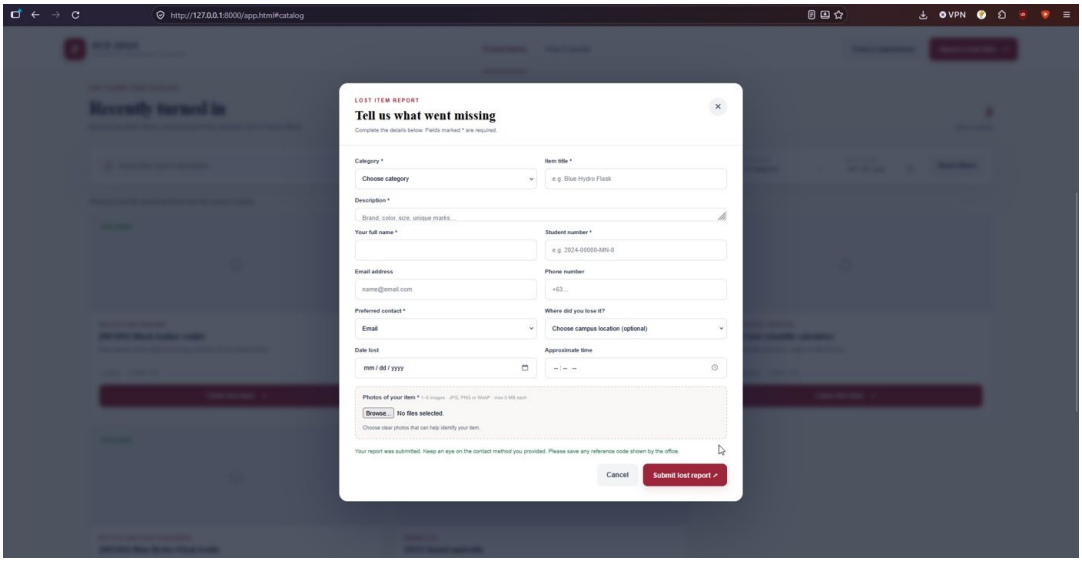


Figure 4. After submission: a success message but no reference code.

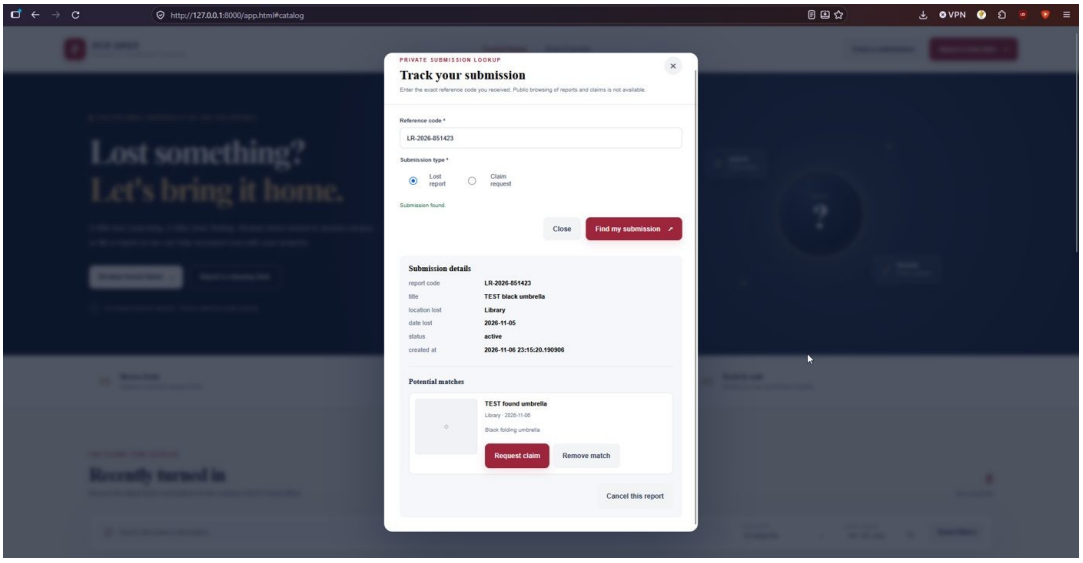


Figure 5. Tracking a lost report by code, with a potential match.

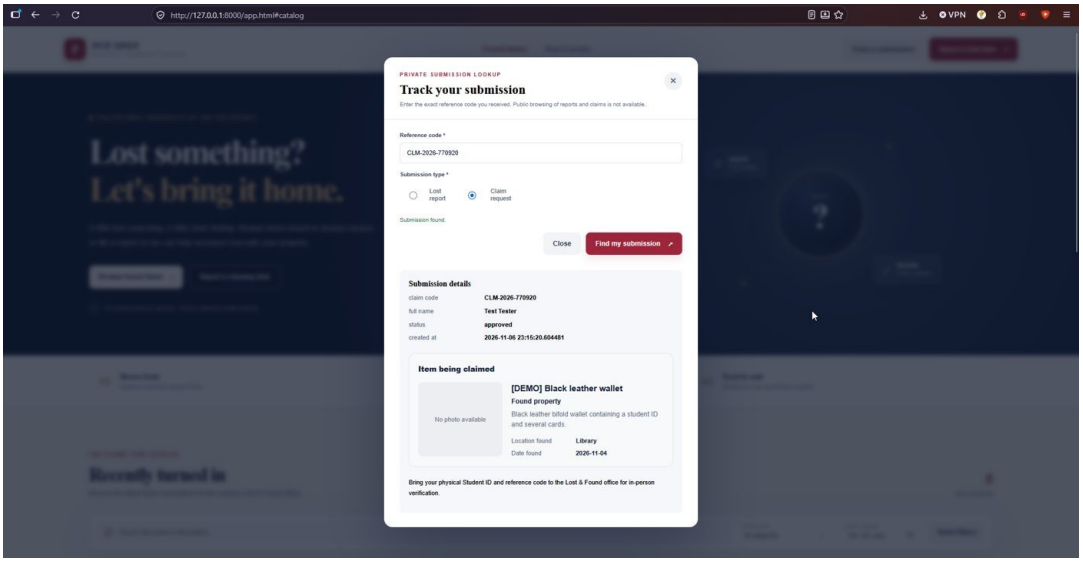


Figure 6. Tracking a claim by code (status approved by the admin API test).

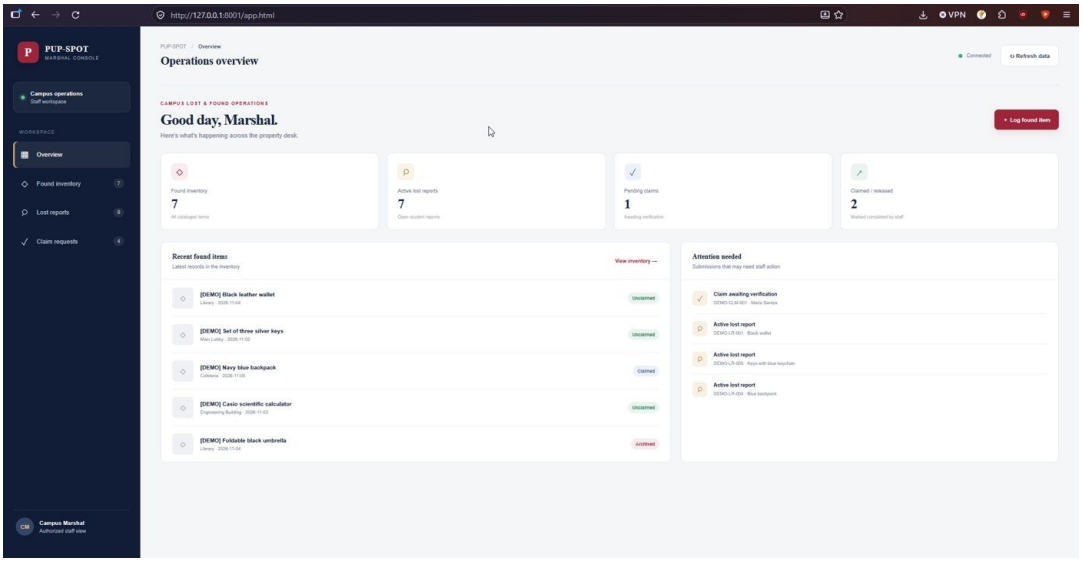


Figure 7. Admin console overview, reached with no login.

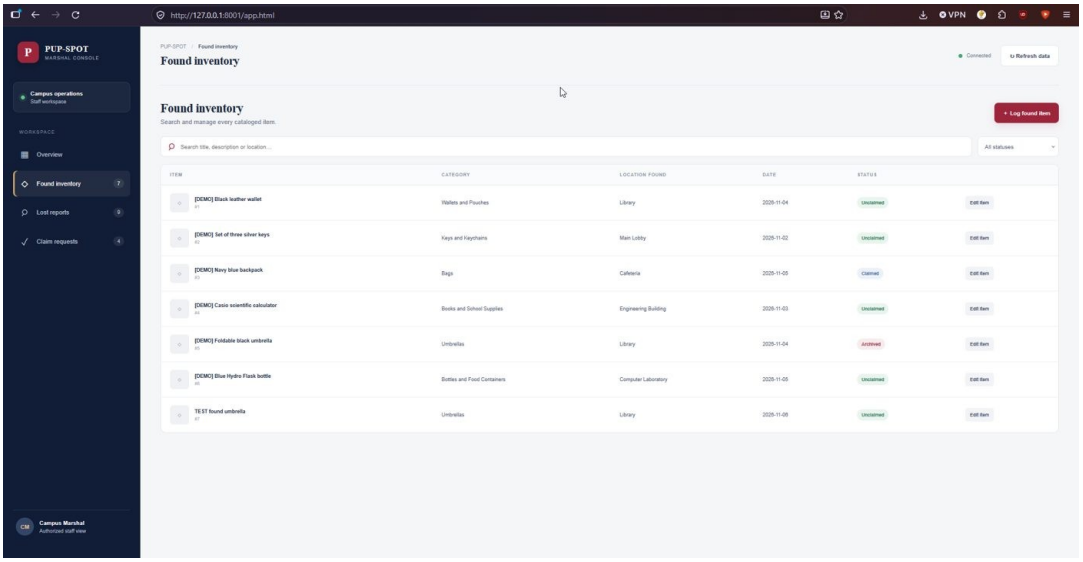


Figure 8. Admin found inventory.

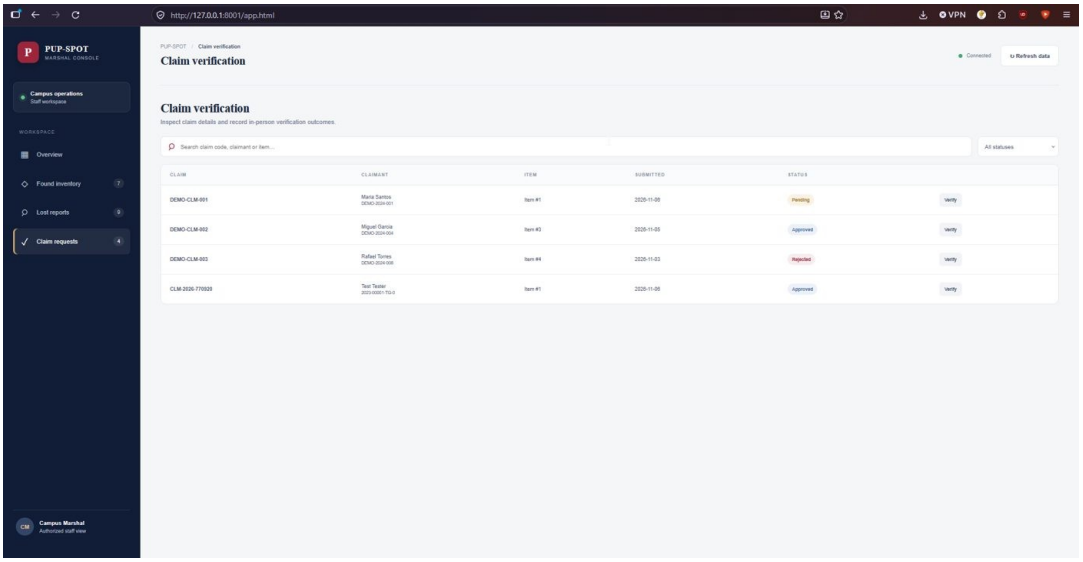


Figure 9. Admin claim verification list.

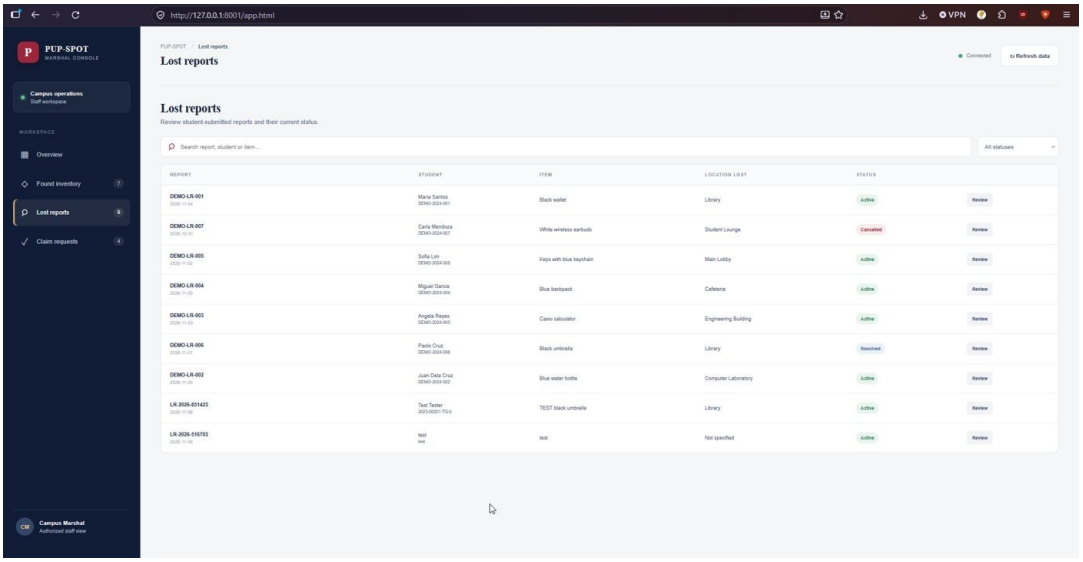


Figure 10. Admin lost reports list.

6. Issues found

#	Issue	Impact and suggested fix
1	When the EmailJS or TextBee keys are not set, PHP prints "Undefined array key" warnings into the API response before the JSON. Report and claim creation then return a body that is not valid JSON	The page cannot read the new reference code, so the student sees no code (Figure 4) and with no notification there is no other way to learn it. The README says these warnings are harmless, which is not true here. Check isset on the keys, turn display_errors off, and return the code in the response
2	POST /api/public/lost-reports/view with an empty body ends in an uncaught TypeError. The response is HTTP 200 and contains the PHP error text with file paths	Information leak and a wrong status code. Validate the body and keep display_errors off

3	The admin console has no login. The README states that the network is the only boundary	Anyone who can reach port 8001 can approve claims and read every student's contact details. Add authentication
4	docs/api.md says the public server connects with the admin role. The code uses lostandfound_public	The documentation is out of date (the code is the safer one). Update the document
5	The photo upload uses a storage folder outside the database	Images are not in a backup of the database (see Activity 6)

What worked well: the README, scripts and demo data ran with no errors on the first try, the demo script can be rerun, and the separation of public and admin roles is real.

7. Feedback sent to owners

Sent on 6 November 2026: issues 1 to 4 with the exact output. The setup guide was otherwise complete.

8. Reproducibility conclusion

Reproducible. A fresh install following the README gave a working system in one pass. The only functional gap is that a student cannot see the reference code without the email or SMS services configured.

AI usage declaration

- Tool: Claude (Anthropic).
- Tasks: interpreting the results of the testing.
- Validation: the commands were run by the group on its own server, and every result in this report comes from the saved output files.